المملكة المغربية
ⵜⴰⴳⵍⴷⵉⵜ ⵏ ⵍⵎⵖⵔⵉⴱ
وزارة التربية الوطنية
والتعليم الأولي والرياضة
ⵜⴰⵎⴰⵡⴰⵙⵜ ⵏ ⵓⵙⴳⵎⵉ ⴰⵏⴰⵎⵓⵔ
ⴷ ⴰⵙⵙⵏⵎⵉⴷ ⴰⵎⵣⵡⴰⵔⵓ ⴷ ⵜⵓⵏⵏⵓⵏⵜ

Examen National du Brevet de Technicien Supérieur (BTS) 2

Session 2026

- Correction -

Filière :	Développement des Applications Informatiques -DAI-	Durée : 2 heures
Épreuve :	Développement Mobile Android	Coefficient : 45

CORRIGÉ

DOSSIER I : UI & ÉVÉNEMENTS

1. Interface

1. XML complet (ConstraintLayout) :

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <EditText
        android:id="@+id/titre"
        android:hint="Titre␣événement"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>

    <CheckBox
        android:id="@+id/important"
        android:text="Important"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintTop_toBottomOf="@id/titre"
        app:layout_constraintStart_toStartOf="parent"/>

    <Button
        android:id="@+id/btnAjouter"
        android:text="Ajouter"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintTop_toBottomOf="@id/important"
        app:layout_constraintStart_toStartOf="parent"/>

    <StackView
```

```
        android:id="@+id/stack"
        android:layout_width="match_parent"
        android:layout_height="300dp"
        app:layout_constraintTop_toBottomOf="@id/btnAjouter"/>

</androidx.constraintlayout.widget.ConstraintLayout>
```

2. Rôle du StackView :
   — Permet d'afficher des éléments sous forme de pile (stack)
   — Un seul élément visible à la fois
   — Navigation possible par geste (swipe)

DOSSIER II : STOCKAGE & SERIALIZABLE

1. Classe Event

```
class Event implements Serializable {
    int id;
    String titre;
    boolean important;
    long date;
}
```

2. Fichiers

1. Sauvegarde liste :

```
ObjectOutputStream oos =
new ObjectOutputStream(openFileOutput("events.dat", MODE_PRIVATE));

oos.writeObject(listeEvents);
oos.close();
```

2. Lecture :

```
ObjectInputStream ois =
new ObjectInputStream(openFileInput("events.dat"));

ArrayList<Event> list = (ArrayList<Event>) ois.readObject();
ois.close();
```

3. SharedPreferences

```
SharedPreferences sp = getSharedPreferences("app", MODE_PRIVATE);

// sauvegarde
sp.edit()
.putInt("nb", 5)
.putBoolean("notif", true)
.apply();

// suppression clé
sp.edit().remove("nb").apply();
```

```
// suppression totale
sp.edit().clear().apply();
```

DOSSIER III : ADAPTER & SQLITE

1. SQLite

```
class DBHelper extends SQLiteOpenHelper {

    public DBHelper(Context c){
        super(c,"db",null,1);
    }

    public void onCreate(SQLiteDatabase db){
        db.execSQL("CREATE␣TABLE␣event(id␣INTEGER␣PRIMARY␣KEY␣AUTOINCREMENT
           ,␣titre␣TEXT,␣important␣INTEGER)");
    }

    public void onUpgrade(SQLiteDatabase db,int o,int n){
     db.execSQL("DROP␣TABLE␣IF␣EXISTS␣event");
26 onCreate(db)
    }
}
```

Opérations :

```
// insert
ContentValues v = new ContentValues();
v.put("titre", e.getTitre());
v.put("important", e.isImportant());
db.insert("event", null, v);

// update
db.update("event", v, "id=?", new String[]{id});

// delete
db.delete("event", "id=?", new String[]{id});

// get all
Cursor c = db.rawQuery("SELECT␣*␣FROM␣event", null);

// get importants
    ArrayList<Event> liste = new ArrayList<>();

    Cursor cursor = db.rawQuery("SELECT␣*␣FROM␣event␣WHERE␣important=?",
       new String[]{"1"});

    if (cursor.moveToFirst()) {
        do {
            Event ev = new Event(
                    cursor.getInt(0),
                    cursor.getString(1),
                    cursor.getInt(2)
            );
```

```
            liste.add(ev);
        } while (cursor.moveToNext());
    }

    cursor.close();
```

2. Adapter personnalisé

```
class EventAdapter extends BaseAdapter {

    Context c;
    List<Event> list;
    public EventAdapter(Context context, ArrayList[] list) {
        this.c = context;
        this.list = list;
    }

    public int getCount(){ return list.size(); }
    @Override
    public Object getItem(int position) {
        return data[position];
    }

    @Override
    public long getItemId(int position) {
        return position;
    }
    public View getView(int i, View v, ViewGroup parent){
    // On peut mettere LayoutInflater.from(c) dans le constructeur de l'
       adapter
    // Vous pouvez trouvez parfois View convertView et parfois View view,
       il  n y  a aucune différence fonctionnelle entre les deux
        if(v == null){
            v = LayoutInflater.from(c).inflate(R.layout.item, parent, false
               );
        }
        TextView t = v.findViewById(R.id.txt);
        t.setText(list.get(i).getTitre());
        return v;
    }
}
```

ViewHolder :
- Permet d'éviter findViewById à chaque fois
- Améliore les performances
- Réutilise les vues

Pourquoi plus performant ?
- Moins d'accès mémoire
- Réutilisation des vues
- Scroll plus fluide

3. Gestion UI

```
// bouton
btnAdd.setOnClickListener(v -> {

    String texte = etText.getText().toString();
    boolean check = etCheck.isChecked();

    Evenement ev = new Evenement(texte, check);

    dao.insert(ev);

    liste.add(ev);

    adapter.notifyDataSetChanged();
});

//checkbox
etCheck.setOnCheckedChangeListener((buttonView, isChecked) -> {
    /Toast.makeText(this,"Màj␣checkbox",Toast.LENGTH_SHORT).show();
});

// clic item
stackView.setOnItemClickListener((parent, view, position, id) -> {

    Evenement ev = liste.get(position);

    String texte;

    if(ev.isImportant()){
        texte = "Evenement␣important";
    } else {
        texte = "Evenement␣non␣important";
    }

    Toast.makeText(this, texte, Toast.LENGTH_SHORT).show();
});

// long click
stackView.setOnItemLongClickListener((parent, view, position, id) -> {

    Evenement ev = liste.get(position);

    dao.delete(ev.getId());

    liste.remove(position);

    adapter.notifyDataSetChanged();

    return true;
});
```

4. Exceptions

```
try {
    int x = Integer.parseInt(edit.getText().toString());
```

```
} catch(NumberFormatException e){
    edit.setError("Erreur");
}

// TextWatcher
edit.addTextChangedListener(new TextWatcher(){
    public void onTextChanged(CharSequence s,int a,int b,int c){
        if(s.length()>20){
            edit.setError("Max␣20␣caractères");
        }
    }
});
```

DOSSIER IV : FIREBASE & MVVM

1. Firebase

```
DatabaseReference ref = FirebaseDatabase.getInstance()
.getReference().child("events");

// ajout
ref.push().setValue(event);

// update
ref.child(id).setValue(event);

// delete
ref.child(id).removeValue();

// lecture
ref.addValueEventListener(new ValueEventListener(){
    public void onDataChange(DataSnapshot ds){
        for(DataSnapshot d: ds.getChildren()){
            Event e = d.getValue(Event.class);
        }
    }
});
```

2. MVVM

- DAO : accès direct aux données
- Repository : couche intermédiaire
- ViewModel : logique métier et communication avec UI

3. Explication globale

- Activity → interface utilisateur
- ViewModel → logique
- Repository → gestion données
- DAO → accès SQLite
- Firebase → stockage distant
- SharedPreferences → configuration